

2º curso / 2º cuatr.

Grados en
Ing. Informática

Arquitectura de Computadores. Ejercicios y cuestiones

Tema 2. Programación paralela

Material elaborado por: Mancia Anguita

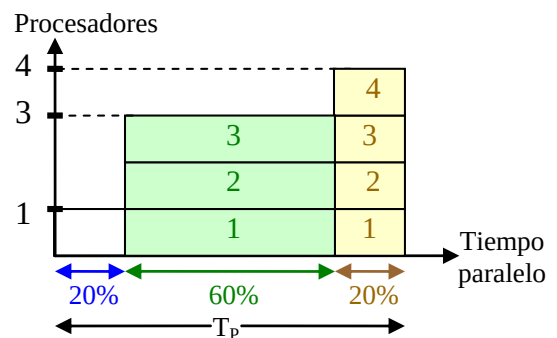
1 Ejercicios

Ejercicio 1. Un programa tarda 40 s en ejecutarse en un multiprocesador. Durante un 20% de ese tiempo se ha ejecutado en cuatro procesadores (core); durante un 60%, en tres; y durante el 20% restante, en un procesador (consideramos que se ha distribuido la carga de trabajo por igual entre los procesadores que colaboran en la ejecución en cada momento, despreciamos sobrecarga). ¿Cuánto tiempo tardaría en ejecutarse el programa en un único procesador? ¿Cuál es la ganancia en velocidad obtenida con respecto al tiempo de ejecución secuencial? ¿Y la eficiencia?

Solución

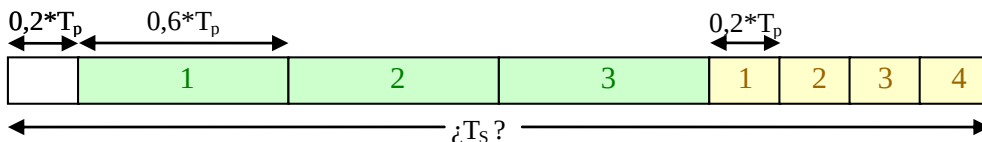
Datos del ejercicio:

En el gráfico de la derecha se representan los datos del ejercicio. Estos datos son la fracción del tiempo de ejecución paralelo (T_p) que supone el código no paralelizable (20% de T_p , es decir, $0,2 \cdot T_p$), la fracción que supone el código paralelizable en 3 procesadores ($0,6 \cdot T_p$) y la que supone el código paralelizable en 4 procesadores ($0,2 \cdot T_p$). Al distribuirse la carga de trabajo por igual entre los procesadores utilizados en cada instante, los trozos asignados a 3 procesadores suponen todos el mismo tiempo (por eso se han dibujado en el gráfico de igual tamaño) e, igualmente, los trozos asignados a 4 procesadores también suponen todos el mismo tiempo.



¿Tiempo de ejecución secuencial, T_s ?:

Los trozos de código representados en la gráfica anterior se deben ejecutar secuencialmente, es decir, uno detrás de otro, como se ilustra el gráfico siguiente:



$$T_s = 0,2 \times T_p + 3 \times 0,6 \times T_p + 4 \times 0,2 \times T_p = (0,2 + 1,8 + 0,8) \times T_p = 2,8 \times T_p = 2,8 \times 40s = 112s$$

¿Ganancia en velocidad, $S(p)$?:

$$S(4) = \frac{T_s}{T_p(4)} = \frac{2,8 \times T_p(4)}{T_p(4)} = 2,8$$

¿Eficiencia, $E(p)$?:

$$E(4) = \frac{S(p)}{p} = \frac{S(4)}{4} = \frac{2,8}{4} = 0,7$$

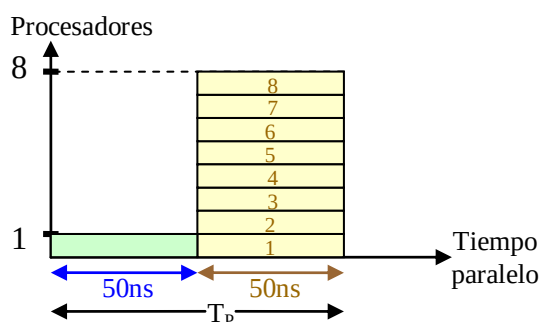


Ejercicio 2. ◀

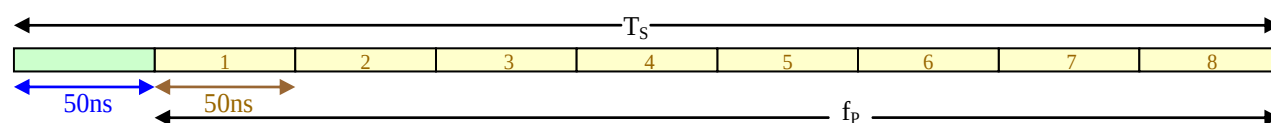
Ejercicio 3. ¿Cuál es fracción de código paralelo de un programa secuencial que, ejecutado en paralelo en 8 procesadores, tarda un tiempo de 100 ns, durante 50ns utiliza un único procesador y durante otros 50 ns utiliza 8 procesadores (distribuyendo la carga de trabajo por igual entre los procesadores)?

Solución

Datos del programa:



¿Fracción de código paralelizable del programa secuencial, f_p ?:



$$f_p = \frac{8 \times 50ns}{T_s} = \frac{8 \times 50ns}{50ns + 8 \times 50ns} = \frac{8}{9}$$



Ejercicio 4. ◀

Ejercicio 5. ◀

Ejercicio 6. ◀

Ejercicio 7. Se quiere paralelizar el siguiente trozo de código:

```
{Cálculos antes del bucle}
for( i=0; i<w; i++) {
    Código para i
}
{Cálculos después del bucle}
```

Los cálculos antes y después del bucle suponen un tiempo de t_1 y t_2 , respectivamente. Una iteración del ciclo supone un tiempo t_i . En la ejecución paralela, la inicialización de p procesos supone un tiempo k_1p (k_1 constante), los procesos se comunican y se sincronizan, lo que supone un tiempo k_2p (k_2 constante); $k_1p + k_2p$ constituyen la sobrecarga.

- Obtener una expresión para el tiempo de ejecución paralela del trozo de código en p procesadores (T_p).
- Obtener una expresión para la ganancia en velocidad de la ejecución paralela con respecto a una

ejecución secuencial (S_p).

- (c) ¿Tiene el tiempo T_p con respecto a p una característica lineal o puede presentar algún mínimo? ¿Por qué? En caso de presentar un mínimo, ¿para qué número de procesadores p se alcanza?

Solución

Datos del ejercicio:

```

{Cálculos antes del bucle}
for( i=0; i<w; i++) {
    Código para i
}
{cálculos después del bucle}

```



Sobrecarga: $T_0 = k_1 p + k_2 p = (k_1 + k_2) p$

Llamaremos t a $t_1 + t_2$ ($= t$) y k a $k_1 + k_2$ ($= k$)

(a) ¿Tiempo de ejecución paralelo, T_p ?

$$T_p(p, w) = t + \left\lceil \frac{w}{p} \right\rceil \times t_i + k \times p$$

w/p se redondea al entero superior

(b) ¿Ganancia en prestaciones, $S(p, w)$?

$$S(p, w) = \frac{T_s}{T_p(p, w)} = \frac{t + w \times t_i}{t + \left\lceil \frac{w}{p} \right\rceil \times t_i + k \times p}$$

(c) ¿Presenta mínimo? ¿Para qué número p ?

$$T_p(p, w) = t + \left\lceil \frac{w}{p} \right\rceil \times t_i + k \times p \quad (1)$$

En la expresión (1), el término englobado en línea continua, o tiempo de cálculo paralelo, tiene tendencia a decrecer con pendiente que va disminuyendo conforme se incrementa p (debido a que p está en el divisor), y el término englobado con línea discontinua o tiempo de sobrecarga ($k \times p$), crece conforme se incrementa p con pendiente k constante. Dado que la pendiente de la sobrecarga es constante y que la pendiente del tiempo de cálculo decrece conforme se incrementa p , llega un momento en que el tiempo de ejecución en paralelo pasa de decrecer a crecer.

Se puede encontrar el mínimo analíticamente igualando a 0 la primera derivada de T_p . De esta forma se obtiene los máximos y mínimos de una función continua. Para comprobar si en un punto encontrado de esta forma hay un máximo o un mínimo se calcula el valor de la segunda derivada en ese punto. Si, como resultado de este cálculo, se obtiene un valor por encima de 0 hay un mínimo, y si, por el contrario, se obtiene un valor por debajo de 0 hay un máximo. Para realizar el cálculo se debe eliminar el redondeo de la expresión (1) (más abajo se comentará la influencia del redondeo en el cálculo del mínimo):

$$T_p(p, w) = t + \frac{w}{p} \times t_i + k \times p$$

$$T'_p(p, w) = 0 - \frac{w}{p^2} \times t_i + k \Rightarrow -\frac{w}{p^2} \times t_i + k = 0 \Rightarrow p = \sqrt{\frac{w \times t_i}{k}} \quad (2)$$

El resultado negativo de la raíz se descarta. En cuanto al resultado positivo, debido al redondeo, habrá que comprobar para cuál de los naturales próximos al resultado obtenido (incluido el propio resultado si es un número natural) se obtiene un menor tiempo.

En cualquier caso, el número de procesadores que obtengamos debe ser menor que w (que es el grado de paralelismo del código).

La segunda derivada permitirá demostrar que se trata de un mínimo:

$$T_p(p, w) = t + \frac{w}{p} \times t_i + k \times p$$

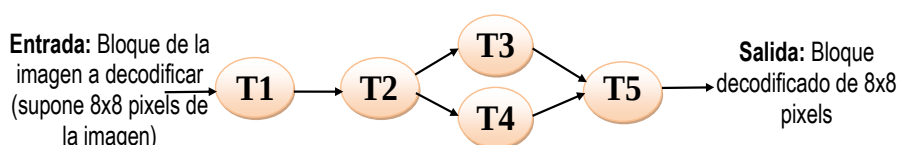
$$T''_p(p, w) = + \frac{2 \times p \times w \times t_i}{p^4} + 0 = \frac{2 \times w \times t_i}{p^3} > 0$$

La segunda derivada es mayor que 0, ya que w , p y t_i son mayores que 0; por tanto, hay un mínimo.



Ejercicio 8. ◀

Ejercicio 9. Se va a paralelizar un decodificador JPEG en un multiprocesador. Se ha extraído para la aplicación el siguiente grafo de tareas que presenta una estructura segmentada (o de flujo de datos):



La tareas 1, 2 y 5 se ejecutan en un tiempo igual a t , mientras que las tareas 3 y 4 suponen $1,5t$

El decodificador JPEG aplica el grafo de tareas de la figura a bloques de la imagen, cada uno de 8x8 píxeles. Si se procesa una imagen que se puede dividir en n bloques de 8x8 píxeles, a cada uno de esos n bloques se aplica el grafo de tareas de la figura. Obtenga la mayor ganancia en prestaciones que se puede conseguir paralelizando el decodificador JPEG en (suponga despreciable el tiempo de comunicación/sincronización): **(a)** 5 procesadores, y **(b)** 4 procesadores. En cualquier de los dos casos, la ganancia se tiene que calcular suponiendo que se procesa una imagen con un total de n bloques de 8x8 píxeles.

Solución

Para obtener la ganancia se tiene que calcular el tiempo de ejecución en secuencial para un tamaño del problema de n bloques $T_s(n)$ y el tiempo de ejecución en paralelo para un tamaño del problema de n y los p procesadores indicados en **(a)** y **(b)** $T_p(p, n)$.

Tiempo de ejecución secuencial $T_s(n)$. Un procesador tiene que ejecutar las 5 tareas para cada bloque de 8x8 píxeles. El tiempo que dedica el procesador a cada bloque es de $3t$ (tareas 1, 2 y 5) + $3t$ (tareas 2 y 4) = $6t$, luego para n bloques el tiempo de ejecución secuencial será el siguiente:

$$T_s = n \times (6t)$$

(a) Tiempo de ejecución paralelo y ganancia en prestaciones para 5 procesadores. $T_p(5, n)$, $S(5, n)$. Cada tarea se asigna a un procesador distinto, por tanto todas se pueden ejecutar en paralelo (ver asignación de tareas a procesadores en la Tabla 1). El tiempo de ejecución en paralelo en un pipeline consta de dos tiempos: el tiempo que tarda en procesarse la primera entrada (ver celdas con fondo verde en la tabla) + el tiempo que tardan cada uno del resto de bloques (entradas al cauce) en terminar. Este último depende de la etapa más lenta, que en este caso es la etapa 3 ($1,5t$ frente a t en el resto de etapas).

$$T_p(5, n) = 4,5t + (n - 1) \times 1,5t = 3t + n \times 1,5t$$

$S(5, n) = \frac{T_s(n)}{T_p(5, n)} = \frac{n \times 6t}{3t + n \times 1,5t} \xrightarrow{n \rightarrow \infty} 4$ La ganancia se aproxima a 4 para n suficientemente grande.

Tabla 1. Asignación de tareas a procesadores, ocupación de las etapas del cauce para los primeros bloques procesados y tiempo de procesamiento del primer bloque y de los siguientes para 5 procesadores. Si el almacenamiento de los resultados de una etapa que necesita como entrada la siguiente es un problema en algún momento (por no haber almacenamiento disponible), acabarán entrando los bloques en P1 cada 1,5t, es decir, a la misma velocidad a la que salen del cauce.

Asignación de tareas a 5 procesadores P1, P2, P3, P4 y P5					
Etapas 1 (t)	Etapas 2 (t)	Etapas 3 (1,5t)	Etapas 4 (t)	Terminado	Tiempo procesa.
T1 (procesador P1)	T2 (procesador P2)	T3 (procesador P3) y T4 (procesador P4)	T5 (procesador P5)		
0 - Bloque 1 - t					Procesamiento Bloque 1 (4,5t)
t - Bloque 2 - 2t	t - Bloque 1 - 2t				
2t - Bloque 3 - 3t	2t - Bloque 2 - 3t	2t - Bloque 1 - 3,5t			
3t - Bloque 4 - 4t	3t - Bloque 3 - 4t	3,5t - Bloque 2 - 5t	3,5t - Bloque 1 - 4,5t		
4t - Bloque 5 - 5t	4t - Bloque 4 - 5t	5t - Bloque 3 - 6,5t	5t - Bloque 2 - 6t	Bloque 1	$t + t + 1,5t + t = 4,5t$
5t - Bloque 6 - 6t	5t - Bloque 5 - 6t	6,5t - Bloque 4 - 8t	6,5t - Bloque 3 - 7,5t	Bloque 2	$4,5t + 1,5t = 6t$
6t - Bloque 7 - 7t	6t - Bloque 6 - 7t	8t - Bloque 5 - 9,5t	8t - Bloque 4 - 9t	Bloque 3	$6t + 1,5t = 7,5t$
...	...	...	...	Bloque 4	$7,5t + 1,5t = 9t$
$T_{\text{entrada_etapa}} - \text{Bloque } x - T_{\text{salida_etapa}}$					

(b) Tiempo de ejecución paralelo y ganancia en prestaciones para 4 procesadores $T_p(4, n)$, $S(4, n)$. Se deben asignar las 5 tareas a los 4 procesadores de forma que se consiga el mejor tiempo de ejecución paralelo, es decir, el menor tiempo por bloque una vez procesado el primer bloque. Una opción que nos lleva al menor tiempo por bloque consiste en unir la Etapa 1 y 2 del pipeline del caso (a) en una única etapa asignando T1 y T2 a un procesador (ver asignación de tareas a procesadores en la Tabla 2). Con esta asignación la etapa más lenta supone 2t (Etapa 1); habría además una etapa de 1,5 t (Etapa 2) y otra de t (Etapa 3). Si, por ejemplo, se hubieran agrupado T3 y T4 un procesador la etapa más lenta supondría 3t.

$$T_p(4, n) = 4,5t + (n - 1) \times 2t = 2,5t + n \times 2t$$

$S(4, n) = \frac{T_s(n)}{T_p(4, n)} = \frac{n \times 6t}{2,5t + n \times 2t} \xrightarrow{n \rightarrow \infty} 3$ La ganancia se aproxima a 3 para n suficientemente grande.

Tabla 2. Asignación de tareas a procesadores, ocupación de las etapas del cauce para los primeros bloques procesados y tiempo de procesamiento del primer bloque y de los siguientes para 4 procesadores. Si el almacenamiento de los resultados de una etapa que necesita como entrada la siguiente es un problema en algún momento (por no haber almacenamiento disponible), acabarán entrando los bloques en P1 cada 1,5t, es decir, a la misma velocidad a la que salen del cauce.

Possible assignment of tasks to 4 processors P1, P2, P3 and P4				
Etapa 1 (2t)	Etapa 2 (1,5t)	Etapa 3 (t)	Terminado	Tiempo procesamiento
T1 y T2 (procesador P1)	T3 (procesador P2) y T4 (procesador P3)	T5 (procesador P4)		
0 - Bloque 1 - 2t				Procesamiento Bloque 1 (4,5t)
2t - Bloque 2 - 4t	2t - Bloque 1 - 3,5t			
4t - Bloque 3 - 6t	4t - Bloque 2 - 5,5t	3,5t - Bloque 1 - 4,5t		
6t - Bloque 4 - 8t	6t - Bloque 3 - 7,5t	5,5t - Bloque 2 - 6,5t	Bloque 1	2t + 1,5t + t = 4,5t
8t - Bloque 5 - 10t	8t - Bloque 4 - 9,5t	7,5t - Bloque 3 - 8,5t	Bloque 2	4,5t + 2t = 6,5t
10t - Bloque 6 - 12t	10t - Bloque 5 - 11,5t	9,5t - Bloque 4 - 10,5t	Bloque 3	6,5t + 2t = 8,5t
12t - Bloque 7 - 14t	12t - Bloque 6 - 13,5t	11,5t - Bloque 5 - 12,5t	Bloque 4	8,5t + 2t = 10,5t
...	...	...	Bloque 5	10,5t + 2t = 12,5t
T_entrada_etapa – Bloque x – T_salida_etapa				



Ejercicio 10. Se quiere implementar un programa paralelo para un multicomputador que calcule la siguiente expresión para cualquier x (es el polinomio de interpolación de Lagrange): $P(x) = \sum_{i=0}^n (b_i \cdot L_i(x))$, donde

$$L_i(x) = \frac{(x - a_0) \cdot \dots \cdot (x - a_{i-1}) \cdot (x - a_{i+1}) \cdot \dots \cdot (x - a_n)}{k_i} = \frac{\prod_{\substack{j=0 \\ j \neq i}}^n (x - a_j)}{k_i} \quad i = 0, 1, \dots, n$$

$$k_i = (a_i - a_0) \cdot \dots \cdot (a_i - a_{i-1}) \cdot (a_i - a_{i+1}) \cdot \dots \cdot (a_i - a_n) = \prod_{\substack{j=0 \\ j \neq i}}^n (a_i - a_j) \quad i = 0, 1, \dots, n$$

Inicialmente k_i , a_i y b_i se encuentran en el nodo i y x en todos los nodos. Sólo se van a usar funciones de comunicación colectivas. Indique cuál es el número mínimo de funciones colectivas que se pueden usar, cuáles serían y en qué orden se utilizarían y para qué se usan en cada caso.

Solución

Los pasos ((1) a (5)) del algoritmo para $n=3$ y un número de procesadores de $n+1$ serían los siguientes:

Pr.	Situación Inicial				(1) Resta paralela $A_i=(x-a_i)$ ($(x-a_i)$ se obtiene en P_i)	(2) Todos reducen A_i con resultado en B_i : $B_i = \prod_{i=0}^n A_i$
P0	a_0	x	k_0	b_0	$(x-a_0)$	$(x-a_0).(x-a_1).(x-a_2).(x-a_3)$
P1	a_1	x	k_1	b_1	$(x-a_1)$	$(x-a_0).(x-a_1).(x-a_2).(x-a_3)$
P2	a_2	x	k_2	b_2	$(x-a_2)$	$(x-a_0).(x-a_1).(x-a_2).(x-a_3)$
P3	a_3	x	k_3	b_3	$(x-a_3)$	$(x-a_0).(x-a_1).(x-a_2).(x-a_3)$

Pr.	(3) Cálculo de todos los $L_i(x)$ en paralelo $L_i = B_i / (A_i \cdot k_i)$ ($L_i(x)$ se obtiene en P_i)	(4) Cálculo en paralelo $C_i = b_i \cdot L_i$ ($b_i \cdot L_i(x)$ se obtiene en P_i)	(5) Reducción del contenido de C_i con resultado en P0 ($P(x)$ se obtiene en P0)
P0	$(x-a_1).(x-a_2).(x-a_3) / k_0$	$b_0 \cdot (x-a_1).(x-a_2).(x-a_3) / k_0$	$P = \sum_{i=0}^n (C_i) = \sum_{i=0}^n (b_i \times L_i)$
P1	$(x-a_0).(x-a_2).(x-a_3) / k_1$	$b_1 \cdot (x-a_0).(x-a_2).(x-a_3) / k_1$	
P2	$(x-a_0).(x-a_1).(x-a_3) / k_2$	$b_2 \cdot (x-a_0).(x-a_1).(x-a_3) / k_2$	
P3	$(x-a_0).(x-a_1).(x-a_2) / k_3$	$b_3 \cdot (x-a_0).(x-a_1).(x-a_2) / k_3$	

Como se puede ver en el trazado del algoritmo para $n=3$ mostrado en las tablas, se usan un total de 2 funciones de comunicación colectivas (pasos (2) y (5) en la tabla). En el paso (2) del algoritmo se usa una operación de “todos reducen” para obtener en todos los procesadores los productos de todas las restas $(x-a_i)$. En el paso (5) y último se realiza una operación de reducción para obtener las sumas de todos los productos $(b_i \times L_i)$ en el proceso 0.



Ejercicio 11. (a) Escriba un programa secuencial con notación algorítmica (podría escribirlo en C) que determine si un número de entrada, x , es primo o no. El programa imprimirá si es o no primo. Tendrá almacenados en un vector, NP , los M números primos entre 1 y el máximo valor que puede tener un número de entrada al programa.

(b) Escriba una versión paralela del programa anterior para un multicomputador usando un estilo de programación paralela de paso de mensajes. El proceso 0 tiene inicialmente el número x y el vector NP en su memoria e imprimirá en pantalla el resultado. Considere que la herramienta de programación ofrece funciones `send()/receive()` para implementar una comunicación uno-a-uno asíncrona, es decir, con función `send(buffer, count, datatype, idproc, group)` no bloqueante y `receive(buffer, count, datatype, idproc, group)` bloqueante (Lección 2/Tema 1). En las funciones `send()/receive()` se especifica:

- `group`: identificador del grupo de procesos que intervienen en la comunicación.
- `idproc`: identificador del proceso al que se envía o del que se recibe.
- `buffer`: dirección a partir de la cual se almacenan los datos que se envían o los datos que se reciben.
- `datatype`: tipo de los datos a enviar o recibir (entero de 32 bits, entero de 64 bits, flotante de 32 bits, flotante de 64 bits, ...).
- `count`: número de datos a transferir de tipo `datatype`.

Solución

(a) Programa secuencial que determina si x es o no primo

Pre-condición

x : número de entrada $\{2, \dots, \text{MAX_INPUT}\}$. MAX_INPUT : máximo valor de la entrada

M : número de primos entre 2 y MAX_INPUT (ambos incluidos).

NP : vector con M componentes (los M nº primos desde 2 hasta MAX_INPUT estarán ubicados desde $\text{NP}[0]$ hasta $\text{NP}[M-1]$)

Pos-condición

Imprime en pantalla si el número x es primo o no.

Código

```
if (x>NP[M-1]) {
    print("%u supera el máximo primo a detectar %u \n", x, NP[M-1]);
    exit(1);
}

for (i=0; NP[i]<x;i++) {};

if (x==NP[i])
    printf("%u ES primo\n", x);
else printf("%u NO es primo\n", x);
```

El número de operaciones a realizar depende de en qué posición, n , de NP se encuentre el primer número primo mayor o igual a x . Se puede obtener una versión más eficiente buscando primos que dividan al número de entrada (si los hay, el número no es primo). Se podrían usar algoritmos de búsqueda para mejorar el orden de complejidad.

(b) Programa paralelo para multicomputador que determina si x es o no primo

Se van a repartir las iteraciones del bucle entre los procesadores del grupo. Todos los procesos ejecutan el mismo código

Pre-condición

x : número de entrada $\{2, \dots, \text{MAX_INPUT}\}$; MAX_INPUT : máximo valor de la entrada

M : número de primos entre 2 y MAX_INPUT (ambos incluidos)

grupo: identificador del grupo de procesos que intervienen en la comunicación.

num_procesos: número de procesos en grupo.

idproc: identificador del proceso (dentro del grupo de num_procesos procesos) que ejecuta el código

tipo: tipo de los datos a enviar o recibir

NP : vector con $M + \text{num_procesos}$ componentes. Los nº primos entre 2 y MAX_INPUT estarán en las M primeras posiciones del vector.

b , baux: variables que podrán tomar dos valores 0 (=false) o 1 (=true, si se localiza x en la lista de números primos)

Pos-condición

Imprime en pantalla si el número x es primo o no

Código

```
//Difusión de x y NP
if (idproc==0)
    for (i=1; i<num_procesos; i++) {
        send(NP,M,tipo,i,grupo);
        send(x,1,tipo,i,grupo);
    }
else {
    receive(NP,M,tipo,0,grupo);
    receive(x,1,tipo,0,grupo);
}
```



```
//Cálculo paralelo, asignación estática
NP[M+idproc]=x+1;
For (i=0; NP[i]<x; i=i+num_procesos) {};
b=(x==NP[i])?1:0;

//Comunicación resultados para que pueda imprimir el proceso 0
if (idproc==0)
    for (i=1; i<num_procesos; i++) {
        receive(baux,1, tipo, i, grupo);
        b = b | baux;
    }
else send(b,1, tipo, 0, grupo);

//Proceso 0 imprime resultado
if (idproc==0)
    if (b) printf("%u ES primo", x);
    else printf("%u NO es primo o supera al máximo número primo a detectar", x);
```

El número de instrucciones de comparación en la parte de cálculo paralelo depende de en qué posición, n , de NP se encuentre el primer número primo mayor o igual que x . Se recorre hasta n entre todos los procesos, pero cada uno accede a un subconjunto distinto de componentes del vector entre 0 y n . En particular acceden a $n/\text{num_procesos}$ componentes si num_procesos divide a n , en caso contrario, algunos procesos ejecutarán una iteración más que el resto.



Ejercicio 12.



Ejercicio 13.



Ejercicio 14.

